FAMILY BLOOM

10 VẤN ĐỀ HIỆU NĂNG & KIẾN TRÚC ĐÃ CHỐT

Architecture Spec + Checklist theo dõi / test
Ngày chốt: 10/10/2026

MỤC TIÊU Không “che” lag bằng animation. Giảm tận gốc render cascade, work ở hidden surfaces, bootstrap contention, feed/image decode, graph/calendar recomputation và global UI micro-render.

Tóm tắt điều hành

- Kiến trúc mới ưu tiên cache-first, normalized stores, selector nhỏ và patch đúng entity thay đổi.
- Persistent UI được giữ để quay lại nhanh nhưng hidden surface phải WARM/SUSPENDED và gần như không tiêu CPU.
- Animation / pan / zoom / tab / overlay chạy trên UI thread; network và sync không nằm trên critical transition path.
- Mỗi feature có ownership rõ ràng; root chỉ là global kernel tối thiểu, navigation chỉ quản vị trí người dùng.
- Mọi refactor phải chứng minh bằng test hành vi + render containment, không chỉ “cảm giác mượt hơn”.

```
Target mental model:

Firebase / Socket
      ↓
Repository
      ↓
Normalized Store + Cache
      ↓
Projection / Selector
      ↓
Visible Component Only

Interaction > Visible > Sync > Prefetch > Idle
```

Ma trận 10 vấn đề

#	Vấn đề	Giải pháp chính	Ưu tiên
1	Dữ liệu / Firebase listeners / cache-first	Thu nhỏ root thành global kernel; cache-first + lightweight invalidation; feature store normalized.	**P0**
2	Adaptive Surface Lifecycle	Persistent tabs nhưng hidden surfaces phải ngủ sâu: ACTIVE / WARM / SUSPENDED + scheduler + freeze barrier.	**P0**
3	Moments / Feed / Image Pipeline	Normalized feed + FeedProjection + virtualization + image size buckets/caching.	**P1**
4	Family Tree / Graph Engine	Tách Tree thành subsystem: normalized data, topology/layout cache, 2D virtualization, LOD, UI-thread camera.	**P1**
5	Home / Dashboard Projection	Home là projection trên feature stores, không phải god consumer.	**P1**
6	Planner / Calendar Index	EventStore normalized → CalendarIndex → MonthProjection → DaySummary + Agenda island.	**P1**
7	Startup / Bootstrap Contention	Bootstrap theo stage: shell → identity → cache first paint →	**P0**

#	Vấn đề	Giải pháp chính	Ưu tiên
		critical sync → realtime → deferred → idle.	
8	Overlay / Modal / Toast / Challenge / App Chrome	Overlay layer độc lập khỏi surfaces; store nhỏ theo loại + coordinator priority/dedupe/scope.	**P1**
9	Realtime Signal Containment	Presence / unread / activity / challenge thành signal stores nhỏ, coalesced, batched, selector hẹp.	**P0**
10	Navigation / Route Ownership	Phân biệt Persistent Surface / Transient Route / Global Overlay; navigation không sở hữu dữ liệu.	**P0**

1. Dữ liệu / Firebase listeners / cache-first

Thu nhỏ root thành global kernel; cache-first + lightweight invalidation; feature store normalized.

NGUYÊN TẮC CHỐT Tối ưu phải chặn work/render từ nguồn. Không dùng “if (!focused) return” hoặc animation để che nguyên nhân gốc.

Kiến trúc / cách xử lý

- Root chỉ giữ uid, activeFamilyId, membership tối thiểu, foreground/background và realtime toàn app thật sự cần thiết.
- Feature data không còn full listener ở root; Firebase/Socket đi qua Repository → normalized Store/Cache → selector → component.
- Cache chia theo family + feature, versioned; không dùng một JSON AsyncStorage khổng lồ.
- Lightweight sync manifest/version chỉ markDirty(feature); hidden feature không tự fetch/render.
- Moments: startup cache only, tab active mới delta sync; detail mới temporary realtime.
- Planner/Event: cache-first nhưng sync hot range (Today/Tomorrow/next ~7 days) chủ động hơn.
- Members normalized by uid; một member đổi chỉ component dùng member đó update.

Đánh đổi / hạn chế

Độ phức tạp data layer tăng; cache có thể stale ngắn hạn; invalidation/version phải chính xác.

Checklist test

Mã	Kịch bản test	Kỳ vọng
1A	Khởi động lần 2 khi cache đã có, tắt mạng sau khi mở	Home/feature cache xuất hiện ngay; không full-screen loading.
1B	Thiết bị B thêm Moment, thiết bị A đang ở Home	A chỉ mark Moments dirty; Home không render nếu projection không đổi.
1C	Chuyển Family A → B → A	Mỗi family swap cache riêng gần như tức thì; task generation cũ không đẩy UI sai family.

2. Adaptive Surface Lifecycle

Persistent tabs nhưng hidden surfaces phải ngủ sâu: ACTIVE / WARM / SUSPENDED + scheduler + freeze barrier.

NGUYÊN TẮC CHỐT Tối ưu phải chặn work/render từ nguồn. Không dùng “if (!focused) return” hoặc animation để che nguyên nhân gốc.

Kiến trúc / cách xử lý

- Có SurfaceLifecycleManager chung; không dựa vào useIsFocused riêng lẻ.
- ACTIVE chạy interaction/sync cần thiết; WARM giữ visual/cache/scroll nhưng dừng phần lớn work; SUSPENDED gần 0 CPU.
- Tách Surface Snapshot khỏi Live Engine để giữ trải nghiệm nhưng giải phóng engine nặng.
- BloomTaskScheduler ưu tiên INTERACTION > VISIBLE > SYNC > PREFETCH > IDLE; interaction có quyền cancel prewarm.
- Freeze Barrier + dirtyVersion: hidden surface không nhận React update liên tục, chỉ patch snapshot cuối khi active lại.
- Generation/epoch token chống callback cũ đánh thức UI đã rời.
- Memory budget/cost profile quyết định tab nào warm/suspend, không timer cứng.

Đánh đổi / hạn chế

Dùng thêm RAM cho snapshot/warm surface; lifecycle phức tạp hơn; resume phải staged để tránh burst.

Checklist test

Mã	Kịch bản test	Kỳ vọng
2A	Đứng Home 10 giây không thao tác	Moments/Planner/Tree không render/derive vô lý; SUSPENDED gần 0 CPU.
2B	Home → Moments → Home liên tục	Giữ scroll/snapshot; không refetch/remount toàn feature.
2C	Đang có Chess nhưng rời board	Socket/model vẫn đúng; board/FX/hints không repaint khi hidden.

3. Moments / Feed / Image Pipeline

Normalized feed + FeedProjection + virtualization + image size buckets/caching.

NGUYÊN TẮC CHỐT Tối ưu phải chặn work/render từ nguồn. Không dùng “if (!focused) return” hoặc animation để che nguyên nhân gốc.

Kiến trúc / cách xử lý

- MomentRepository → MomentStore → FeedProjection → VisibleRows → MomentCard(id) → ImagePipeline.
- Entity patch theo id; reaction/comment summary không làm cả feed reconcile.
- FeedProjection quản lý order/filter/pagination/new-item queue; new items không tự đẩy người đọc khỏi vị trí.
- Virtualization tune theo card/media/device, stable key = momentId.
- Scroll restore dùng anchorMomentId + offset, không chỉ scrollY.
- Ảnh tách avatar/feed/viewer; Cloudinary size bucket cố định; request dedupe.
- Visible → memory cache; nearby → prefetch; far → disk only. Video: active/prepared/released theo khoảng cách viewport.

Đánh đổi / hạn chế

Normalized store và media lifecycle phức tạp; prefetch sai có thể tốn network/RAM; virtualization quá gắt có thể blank.

Checklist test

Mã	Kịch bản test	Kỳ vọng
3A	Scroll feed dài 100+ Moment rồi quay lại	Không càng scroll càng lag; anchor restore đúng.
3B	Reaction vào Moment không nằm trên viewport	Không render cả feed/visible cards không liên quan.
3C	Mở ảnh trong feed rồi viewer	Feed dùng bản vừa đủ; viewer mới lấy bản lớn; RAM không tăng mãi.

4. Family Tree / Graph Engine

Tách Tree thành subsystem: normalized data, topology/layout cache, 2D virtualization, LOD, UI-thread camera.

NGUYÊN TẮC CHỐT Tối ưu phải chặn work/render từ nguồn. Không dùng “if (!focused) return” hoặc animation để che nguyên nhân gốc.

Kiến trúc / cách xử lý

- TreeRepository → TreeStore → GraphModel → GraphRenderer.
- TreeStore chia Person/Relationship/Topology/Layout/Viewport/Camera/Selection slices.
- Phân biệt visual/data change và structural/topology change; avatar/name không được chạy lại graph layout.
- Layout cache theo topologyVersion; hướng tới recompute affected branch thay vì toàn graph.
- 2D spatial virtualization: chỉ viewport + overscan có interactive node views.
- Edges batch trong một drawing/SVG/canvas layer khi phù hợp.
- Pan/zoom bằng SharedValue/native transform; React chỉ nhận camera snapshot khi cần.
- LOD theo zoom: far marker → medium avatar/name → near full card.

Đánh đổi / hạn chế

Tree trở thành subsystem riêng, code phức tạp hơn; incremental layout và invalidation cần test kỹ.

Checklist test

Mã	Kịch bản test	Kỳ vọng
4A	Đổi avatar 1 người trong tree 300 người	Chỉ node đó update; topology/layout version không đổi.
4B	Thêm quan hệ cha-con	Topology version tăng; chỉ vùng ảnh hưởng recompute nếu có thể.
4C	Pan/zoom liên tục tree lớn	60fps target; không setState per-frame; node ngoài viewport không có full view/avatar.

5. Home / Dashboard Projection

Home là projection trên feature stores, không phải god consumer.

NGUYÊN TẮC CHỐT Tối ưu phải chặn work/render từ nguồn. Không dùng “if (!focused) return” hoặc animation để che nguyên nhân gốc.

Kiến trúc / cách xử lý

- Moment/Event/Member/Activity/HomeHub stores → HomeProjection → 5–8 render islands độc lập.
- HomeProjection chỉ chứa output nhỏ như latestMomentIds, nextEventId, unreadCount, summary version.
- Mỗi island có selector riêng; feature update chỉ notify khi output Home thực sự đổi.
- Derived filter/sort không chạy trong Home render.
- Cache-first startup, stale-tolerant cho nội dung không critical; Event/upcoming giữ selective freshness.
- Priority tiers: identity → important event/alert → moments/activity → decorative.
- HomeSnapshot giữ visible state/scroll/currentFamily/projectionVersion; quay lại paint snapshot rồi patch.

Đánh đổi / hạn chế

Nhiều tầng dependency/projection hơn; invalidation selector phải chính xác; island độc lập có thể lệch nhịp nếu không batch transaction.

Checklist test

Mã	Kịch bản test	Kỳ vọng
5A	Reaction ở Moment cũ không hiển thị trên Home	0 Home render.
5B	Next Event thay đổi	Chỉ UpcomingEvent island render.
5C	Badge unread tăng	Chỉ badge/icon cần thiết render; tab shell/home khác đứng yên.

6. Planner / Calendar Index

EventStore normalized → CalendarIndex → MonthProjection → DaySummary + Agenda island.

NGUYÊN TẮC CHỐT Tối ưu phải chặn work/render từ nguồn. Không dùng “if (!focused) return” hoặc animation để che nguyên nhân gốc.

Kiến trúc / cách xử lý

- eventsById + eventsByDay/eventsByMonth/upcomingIds/recurringRules với dayKey/monthKey family timezone.
- MonthProjection rất nhỏ; DayCell không subscribe full Event object.
- DaySummaryStore chỉ chứa eventCount/hasImportant/hasBirthday/hasReminder/dominantType.
- SelectionStore tách riêng; chọn ngày chỉ old/new cell + Agenda update.
- UI chỉ giữ prev/current/next month; data cache có thể nhiều hơn.
- Month slide chạy UI thread, settle rồi mới commit logical month.
- Recurrence chỉ materialize visible range, cache theo ruleVersion+monthKey.
- Event edit invalidate đúng day/month/upcoming range, không invalidate toàn calendar.

Đánh đổi / hạn chế

Invaliation logic/recurrence phức tạp; day/timezone normalization phải tuyệt đối nhất quán.

Checklist test

Mã	Kịch bản test	Kỳ vọng
6A	Đổi title event cùng ngày	Chỉ event card/agenda cần thiết render; DayCell không đổi nếu summary giữ nguyên.
6B	Chuyển event ngày 10 → 12	Chỉ day 10/day 12 + agenda/index liên quan update.
6C	Swipe tháng liên tục	Transition UI-thread; không 42 cells + full EventStore cùng thức dậy.

7. Startup / Bootstrap Contention

Bootstrap theo stage: shell → identity → cache first paint → critical sync → realtime → deferred → idle.

NGUYÊN TẮC CHỐT Tối ưu phải chặn work/render từ nguồn. Không dùng “if (!focused) return” hoặc animation để che nguyên nhân gốc.

Kiến trúc / cách xử lý

- Bootstrap Coordinator chung; không để mỗi provider/useEffect tự startup ở root.
- Không dùng một isAppReady; readiness tách identity/shell/home snapshot/realtime/feature.
- Splash chỉ giữ tới navigation shell + identity decision + minimum visual snapshot.
- Cold start khác foreground resume; resume chỉ hot delta/version check.
- Không Promise.all mọi feature.
- Feature failure isolated: Planner sync fail thì dùng cache, app vẫn usable.
- Mỗi startup task có contract priority/cost/cancel/network/family/blocksFirstPaint.
- Generation token bảo vệ activeFamily đổi giữa bootstrap.

Đánh đổi / hạn chế

Startup coordinator phức tạp hơn; first paint có thể dùng cache cũ ngắn hạn; progressive hydration cần tránh layout jump.

Checklist test

Mã	Kịch bản test	Kỳ vọng
7A	Cold start có cache	Home/cached shell xuất hiện trước secondary sync/prewarm.
7B	Cold start mạng chậm	App vẫn usable bằng cache; feature stale riêng, không kẹt global loading.
7C	Background → foreground	Không chạy lại full bootstrap; chỉ hot delta/version sync.

8. Overlay / Modal / Toast / Challenge / App Chrome

Overlay layer độc lập khỏi surfaces; store nhỏ theo loại + coordinator priority/dedupe/scope.

NGUYÊN TẮC CHỐT Tối ưu phải chặn work/render từ nguồn. Không dùng “if (!focused) return” hoặc animation để che nguyên nhân gốc.

Kiến trúc / cách xử lý

- AppSurfaceLayer tách OverlayLayer; overlay không là con logic của screen hiện tại.
- ToastStore/ModalStore/SheetStore/ChallengeStore/AlertStore riêng; host chỉ subscribe store mình.
- Host persistent, content transient; không remount portal infrastructure mỗi lần.
- Overlay animation UI-thread; blocking overlay khóa interaction/work phụ nhưng không destroy surface.
- OverlayCoordinator quản priority/dedupe/replace/scope/timeout/blocking/generation.
- Keyboard adjustment trong overlay layer, tránh relayout whole app.
- AppChrome có selectors riêng; badge đổi chỉ badge render.
- Overlay action ưu tiên command/id thay stale closure.

Đánh đổi / hạn chế

Queue/priority rule phức tạp; callback/command phải scope đúng session; persistent host dùng thêm chút memory.

Checklist test

Mã	Kịch bản test	Kỳ vọng
8A	Toast xuất hiện khi ở Planner	Chỉ ToastHost render; Planner không render lại.
8B	Challenge tới khi đang ở tab khác	ChallengeHost xuất hiện đúng scope; tab hiện tại không remount.
8C	Keyboard mở trong composer sheet	Sheet tự điều chỉnh; root/tabbar/screen không relayout mạnh.

9. Realtime Signal Containment

Presence / unread / activity / challenge thành signal stores nhỏ, coalesced, batched, selector hẹp.

NGUYÊN TẮC CHỐT Tối ưu phải chặn work/render từ nguồn. Không dùng “if (!focused) return” hoặc animation để che nguyên nhân gốc.

Kiến trúc / cách xử lý

- Realtime sources → signal stores → coalescing/batching → selectors → đúng UI.
- PresenceStore[uid], UnreadStore[feature], ChallengeStore, ActivitySummaryStore độc lập.
- Không dùng một RealtimeContext chứa mọi tín hiệu.
- Presence update nhanh có thể coalesce trạng thái cuối; Activity có thể batch nhỏ.
- Challenge/game state vẫn priority cao, không delay theo batch thông thường.
- Signal hidden/không visible không được đánh thức các feature screens.

Đánh đổi / hạn chế

Coalescing tạo độ trễ rất nhỏ cho tín hiệu không critical; phải phân loại rõ tín hiệu nào critical.

Checklist test

Mã	Kịch bản test	Kỳ vọng
9A	Một member online/offline liên tục	Chỉ indicator dùng uid đó update; app khác không render.
9B	Unread 3 → 4	Chỉ badge tương ứng render.
9C	Activity nhiều event nhanh	UI nhận batch/coalesced summary; không replay micro-render liên tục.

10. Navigation / Route Ownership

Phân biệt Persistent Surface / Transient Route / Global Overlay; navigation không sở hữu dữ liệu.

NGUYÊN TẮC CHỐT Tối ưu phải chặn work/render từ nguồn. Không dùng “if (!focused) return” hoặc animation để che nguyên nhân gốc.

Kiến trúc / cách xử lý

- Persistent Surface: 5 tabs + persistent game surface cần giữ lifecycle.
- Transient Route: Moment detail, Event detail, Settings, editors…
- Global Overlay: toast/challenge/result/sheets/modals.
- Không dùng route cho thứ đáng ra overlay; không dùng overlay cho screen cần back/history/deep-link.
- Mỗi route có ownership contract: owns data/listener/engine/visual state?
- Detail route subscribe store theo ID; back không làm dữ liệu gốc biến mất.
- Navigation chỉ quản “đang nhìn đâu”, không quản “dữ liệu có tồn tại không”.

Đánh đổi / hạn chế

Cần refactor route boundaries/deep-link/back semantics; một số flow hiện tại có thể phải đổi ownership.

Checklist test

Mã	Kịch bản test	Kỳ vọng
10A	Mở Moment detail rồi back	Feed/store giữ nguyên, scroll/anchor không mất.
10B	Challenge khi đang ở Event detail	Challenge là overlay, không push/pop route hoặc phá detail stack.
10C	Chess đang chơi → tab khác → quay lại	Persistent game surface phục hồi snapshot, không recreate board/game state.

Thứ tự refactor đề xuất

QUY TẮC Không rewrite 10 hệ thống cùng lúc. Đặt foundation trước, sau đó migrate từng feature để luôn có build testable.

Phase	Phạm vi	Mục tiêu
Phase A - Core Foundation	Vấn đề 1, 2, 7, 9, 10	Store/cache contracts, SurfaceLifecycleManager, BloomTaskScheduler, BootstrapCoordinator, signal stores, route ownership. Không đổi UX lớn.
Phase B - Global UI containment	Vấn đề 8	OverlayCoordinator + persistent hosts + AppChrome selectors.
Phase C - Home & Planner	Vấn đề 5, 6	HomeProjection/render islands; EventStore/CalendarIndex/MonthProjection/DaySummary.
Phase D - Moments	Vấn đề 3	Normalized MomentStore, FeedProjection, virtualization, image pipeline/cache buckets.
Phase E - Family Tree	Vấn đề 4	Graph subsystem, store slices, topology/layout cache, 2D virtualization, LOD.
Phase F - Hardening	Tất cả	Render-count diagnostics trong Settings developer; memory pressure; cold/warm start; rapid tab/family switching; Chess/Xiangqi regression.

Master acceptance checklist

☐ Cold start có cache: Home/shell xuất hiện trước sync thứ cấp; không full bootstrap dưới splash.
☐ Foreground resume không chạy lại bootstrap.
☐ Bấm nhanh 5 tab: không white flash, không jump-cut, hidden surfaces không render storm.
☐ Đứng yên trên Home: Moments/Planner/Tree gần như không tiêu CPU.
☐ Badge/presence/toast/challenge chỉ render đúng host/icon cần thiết.
☐ Moments dài: scroll không càng lúc càng chậm; update một card không reconcile cả feed.
☐ Planner: chọn ngày chỉ 2 DayCell + Agenda; edit event invalidate đúng ngày/tháng.
☐ Family Tree: avatar đổi không layout lại; pan/zoom không đi qua React per-frame.
☐ Đổi Family A→B→A: cache/snapshot swap nhanh, generation cũ không đẩy UI sai family.
☐ Chess/Xiangqi: server-authoritative/reconnect/away timeout/game result không regression.
☐ Memory không tăng vô hạn khi đi qua nhiều tab/media; SUSPENDED giải phóng view/bitmap nặng.
☐ Tất cả feature failures isolated; một sync lỗi không kéo app về loading/error toàn cục.